Choupo: an open-source chemical process simulator for education and research

Vítor Geraldes Pedro Mendes Miguel Rodrigues

White paper · development line `Choupo-dev` · DRAFT of 21 October 2026

Licence. The prose and figures of this white paper are licensed under the Creative Commons Attribution-ShareAlike 4.0 International licence (CC BY-SA 4.0). Code excerpts and Choupo case files quoted in it remain under the GNU General Public License, version 3 or later (GPL-3.0-or-later), the licence of the software they come from. *Choupo* is a registered trade mark of TalentGround Lda.; the trade mark is not licensed by either licence.

Abstract

Choupo is an open-source (GPL-3.0-or-later) chemical process simulator written in C++17 with no external libraries, built for teaching and for research in areas where breadth-first tools are weak. It is a *glass-box* simulator in a sense we define by what a user can observe: the data a calculation used and where each value came from, the hypotheses it rests on, the equations it solves, the parameters it took and whether they are inside their evidence, the numerical method and its iterations, and the results with their conservation balances – all readable in plain-text files and in the run log, none hidden behind a default. This paper presents the project: its motivation, including its response to the arrival of AI agents that can prepare and run a simulation on a student's behalf; what it models today; its architecture (a case is a directory of dictionaries in which every dimensional value carries a unit, the engine has three layers, every model is registered explicitly, and the browser front-end is a visualiser on the ParaView pattern – the case is always text, written in an editor or with an agent, the engine computes natively or in WebAssembly from the same C++, and the interface shows what was computed and computes nothing itself); how its thermophysical data are curated, with a worked example of a binary-pair regression in which a candidate fits its training data well and fails its held-out test; what its verification and validation can and cannot conclude; four representative examples, among them a comparison with an open-source reference simulator; and its limitations. Convergence and conservation are necessary, not sufficient, for physical adequacy: Choupo is a tool to verify calculations, evaluate models and ground decisions, not to prove a design.

DRAFT

1 Motivation and philosophy

Teaching. A student of process simulation meets, in most programs, converged numbers: a column specified and solved, a recycle that closed, stream tables to four decimals. Whether a missing interaction parameter was set to zero, whether a vapour pressure was evaluated outside the range it was fitted on, whether the reactor's enthalpy reference agrees with the stream table's, whether the recycle converged or merely stopped, is not on the screen. Choupo was written for the other kind of reader: the one who must be able to see every one of those things, because seeing them is the course. The source is small, hand-written C++ a student can open; the run log prints each unit as it is solved, with its model, its K-values and every Newton iteration; every assumption the engine makes is announced where it is made and repeated at the end of the run.

Research. The same transparency serves research in areas where breadth-first simulators carry little: membranes (nanofiltration and reverse osmosis), electrochemical separations, aqueous electrolyte chemistry, detailed bioprocess and solids operations. A researcher adds a transport law or a unit operation by writing one class and one registration line, and every balance, every report and the browser front-end apply to it unchanged. Section 6.3 shows one such extension.

Transparency as a design rule. Three rules follow from the two audiences and they shape everything below. A model is *declared* in the case, never chosen by the engine; a word the engine does not know is refused by name, with the accepted words listed, rather than falling through to a default. A solver aid – an initial guess, a tear estimate, a bound – is declared in the case and the solver reports aloud when it uses one. And a number the engine cannot justify is not printed: an energy balance that lacks a formation enthalpy says so and names the missing datum instead of printing a zero. And nothing announced is announced once only: every extrapolation, estimate, unreviewed record and default the run used rides an advisory log into the result file and is replayed, grouped, at the end of the run, under a separate banner that says whether the problem solved is the problem posed – a declared approximation is recorded there, an unauthorised one is refused.

The response to AI agents. An AI agent can now prepare a Choupo case from a sentence, find measured data in an open archive, fit a model's parameters, run the calculation and summarise the result. This changes what a simulation exercise demonstrates. Obtaining a result no longer shows that a student understood anything; what shows it is the student's formulation of the problem, the hypotheses chosen and defended, the design of the solution, the reading of what the engine reports, and the decision taken on it. Choupo's answer is to keep all of that work open to inspection, reproduction and discussion: the agent's case files are plain text the student reads; the data the agent found carry their source, licence and uncertainty; the fit the agent ran declares its acceptance band before it runs and reports a verdict the student can see fail; the engine's log shows what was computed and what was assumed. The repository carries an entry point for agents (`AGENTS.md`) that points into short reference files under `docs/ai/` – the dictionary grammar, the case layout, the thermodynamics, the unit operations, the components, patterns and pitfalls – which `bin/llmctx` concatenates into one context an assistant is handed before it authors a case. The student is not reduced to an auditor of the agent's work. chooses, designs and decides; the agent takes the clerical part, and the engine makes everything either of them did checkable by the other.

We also see the positive side. A student who no longer spends the week typing dictionaries can explore more alternatives – three activity models instead of one, two column configurations instead of one – can afford more demanding models, and can spend the time saved on physical interpretation: why the nitrate slips through the membrane that holds the chloride, why the first law of a plant misses by three percent and which unit carries the gap. Whether this makes the teaching more effective is a goal to be evaluated, not a result: no classroom measurement exists yet, and we do not claim one. The fair comparison, when it is made, is the same student setting up the same transparent tool with and without an agent, with the learning outcome named in advance.

DRAFT

2 Current scope and intended applications

Table 1 says what Choupo models today. Every count in it is read from a file the repository regenerates from the tree, never typed; the commit that regenerated it is named in the source.

3 Architecture

3.1 A case is a directory of plain-text dictionaries

A case is a directory. `system/` says *how* (control, flowsheet topology, solver options, the outer driver, post-processing); `constant/` says *with what* (the thermophysical system, reactions, case-local data); `0/` holds the complete initial state, one file per stream. The flowsheet dictionary carries topology only; a stream's values live in its own file, and a missing or orphan stream file is a fatal error. Listing 1 is the whole topology and the feed of the first tutorial, an isothermal flash of benzene and toluene.

Units are mandatory. Every dimensional scalar in a dictionary carries a named unit – `P 1 bar;`, `F 100 kmol/h;`, `T 25 degC;` – or a bracket dimension set followed by the SI value, `A_w [-1 2 1 0 0] 1.5e-12;`, for a quantity with no short unit name. The parser converts every value to SI on

Table 1: What Choupo models today (development line; counts from `generated/releaseInventory.json`, regenerated at commit `c402561ad`). The latest release is `Choupo-2608`.

Binaries (5)	`choupoSolve` (steady state, recycles by Newton on the tear streams or Wegstein); `choupoBatch` (batch campaigns driven by a recipe); `choupoSemiContinuous` (a time-integrated flowsheet without control loops: start-up, fed-batch, feed-and-bleed); `choupoCtrl` (the same integration with a control layer); `choupoProps` (property evaluation, scans, consistency tests, parameter fitting, curation dossiers).
Unit operations (52)	Flash and saturation (isothermal, adiabatic, bubble and dew points, phase changer, boiler, condenser); reactors (conversion, equilibrium by named reactions, Gibbs minimisation, CSTR, PFR incl. cooled multitubular, batch and dynamic); distillation (rigorous Wang–Henke and simultaneous MESH, multi-feed, side draws, tray hydraulics; shortcut FUG); absorbers, strippers, liquid–liquid extractors; heat exchangers (rating and design), multi-stream exchangers, evaporators; crystallisers (equilibrium, MSMPR, batch population balance); dryers (spray, solid, convective, evaporative), cooling tower; membranes (spiral-wound NF/RO with solution-diffusion, DSPM-DE and solution-diffusion-electromigration laws; batch diafilter; electrodialysis stack and batch rig; ion exchange); adsorption (fixed-bed breakthrough, PSA, TSA); cyclone, bag filter, pneumatic conveyor; compressor, turbine, pump; mixer, splitter, valve, pipe, storage tank.
Thermodynamics	Activity: ideal, NRTL, Wilson, UNIQUAC, UNIFAC, COSMO-SAC (2002). Equations of state: ideal gas, SRK, Peng–Robinson, PC-SAFT (with association). Vapour pressure: Antoine, Ambrose–Walton, Wagner; heat capacity: polynomial, NASA-7, Rowlinson–Bondi. Aqueous electrolytes: single-salt Pitzer and electrolyte-NRTL for flowsheet VLE; Davies, Pitzer–HMW and Edwards–Pitzer speciation with reactive equilibria, a second liquid and solid phases; IAPWS-IF97 steam. Transport correlations as objects with a validity window and a cited anchor.
Catalogue	604 components, 53 aqueous species, 276 mixture parameter records (NRTL 9, UNIQUAC 3, Henry 206, Pitzer 55, eNRTL 3), 4 materials, 4 membranes, 3 adsorbents, 10 utilities – every value with its primary source and a licence the project may redistribute.
Plant scale	Recycles (declared tears, sequential-modular order validated), sectored plants with a thermodynamic model per sector, energy wires between units, utilities, outer drivers (parameter sweeps, design specifications, optimisation by Nelder–Mead or SQP, parameter estimation), post-processing (equipment sizing, Turton bare-module costing, FCI/TCI, an NPV cash flow with a construction schedule and start-up years, economic sensitivity bars, land priced from the layout's plot area, declared quotes and operating crew, pinch targets, plant layout and pipe routing, GMP interiors).
Corpus	480 runnable tutorial cases, 463 of them with a recorded reference answer, indexed in `docs/tutorials-catalogue.md`; 61 interactive EduTool lessons in the browser across ten disciplines (how Choupo works; modelling and scale-up; thermodynamics; separations and phase equilibria; heat transfer and energy; reaction engineering; hydraulics and control; process design and economics; process safety; flowsheeting and numerics).
Intended for	Undergraduate and graduate process simulation and design; research on membranes, electrodialysis, electrolytes and solids; any use where the computation must be inspectable – not for breadth parity with industrial simulators, which it does not seek.

DRAFT

reading and records its dimensions; an unknown unit suffix is refused where it is read, and a value whose declared dimensions are not the ones the reader expects (a pressure where a temperature is required) is refused by name with both dimension sets printed. A number with no unit never reaches a model.

Listing 1: The first tutorial's topology, `system/flowsheetDict`, and its feed, `0/feed` (case files, GPL-3.0-or-later; file banners omitted).

```
units
(
    {
        name        flash01;
        type        isothermalFlash;
        model       rachfordRice;      // the machine, NAMED (type -> model -> operation)
        in          feed;
        outputs     ( liquid  vapor );
        operation   { T 370.0 K;  P 1.0 bar; }
    }
);
// ************************************************************************* 0/feed
componentMolarFlows { benzene 40 kmol/h;  toluene 60 kmol/h; }
T               370 K;
P               100000 Pa;
```

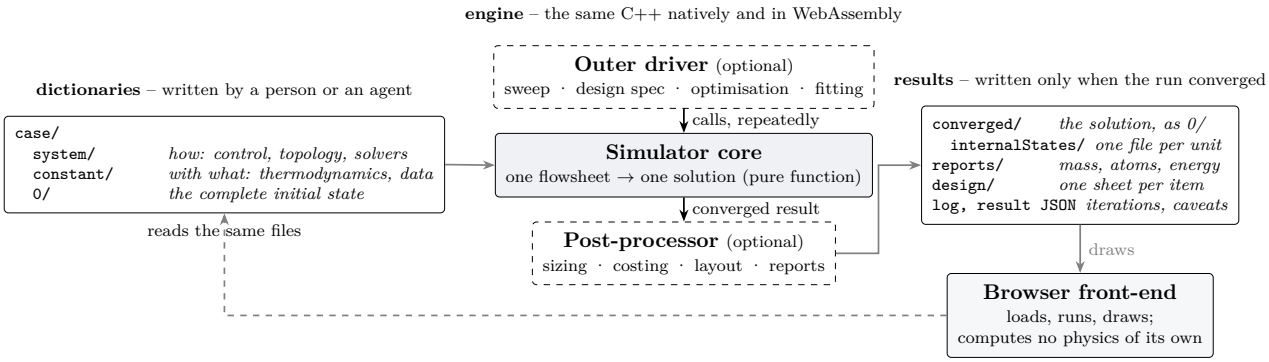


Figure 1: Dictionaries, engine, results and front-end. The three layers of the engine; dashed layers are optional. The front-end runs the same compiled engine in the browser and draws what it published.

3.2 Three layers, explicit factories, state directories

The engine has three layers (Figure 1). The *simulator core* turns one flowsheet into one solution – streams, unit results and key performance indicators – as a pure function of its inputs. An optional *outer driver* calls that function repeatedly: parameter sweeps, design specifications, optimisation, parameter estimation. An optional *post-processor* chain augments a converged solution with equipment sizing, costing, plant layout and reports. Five binaries partition the problem classes (Table 1); numerical strategies coexist inside each binary and are selected in the case files, so there is no binary per algorithm. The whole engine builds with `g++` and GNU make alone – 369 translation units, no external library – so that a student can read every line that computes: the Newton solvers (one- and n-dimensional), Wegstein on declared tear streams, Nelder–Mead, Michelsen's stability test, RK4 and a Rosenbrock23 integrator for stiff dynamics are all in `src/solver/`, every iteration printed at the default verbosity, and convergence is decided by one normalised residual with an OpenFOAM-shaped (`tolerance`, `relTol`, `maxIter`) triple declared per solver. The time-integrated classes share one driver: `choupoBatch` adds a recipe of timed events and two campaign ledgers (material and energy, every integral an exact state difference), `choupoSemiContinuous` a quasi-steady adapter that re-solves any steady unit inside a transient, and `choupoCtrl` PID, schedule and signal controllers with a balance ledger on the integrator's accepted steps.

Explicit factories. Every extensible base class – unit operations, activity models, equations of state, vapour-pressure and heat-capacity models, transport correlations, outer drivers, post-processors, sizers, property operations – has a factory, and every built-in type is registered by one line inside a `registerBuiltins()` function that `main()` calls explicitly. There are no self-registration macros and no static initialisers, so "where does this model come from?" is answered by reading one function. The cost is stated: a new built-in needs one registration line and a rebuild.

A user can write their own unit operations and models. On a local installation, a case may ship its own `code/` directory: a class and one registration line in `registerUserTypes()`, the hook `main()` calls right after the built-ins are registered, compiled against the engine by `bin/buildCode` with a plain, printed `g++` command and linked into a per-case binary. It is the same mechanism the project uses to add a built-in – the same factory, the same explicit line – so a student's unit operation is not a second-class plug-in. Two tutorials do exactly this: `userOp01_yield_reactor` registers a unit operation, `userOp03_case_rate_law` a rate law, which it also writes a second way, as an *expression* in the case's own `constant/reactions` over declared parameters with units, compiled by the engine's formula evaluator with no C++ at all; and any unit may carry a `thermo {}` block that replaces the global thermodynamic models for that unit alone. The hosted site runs the shipped engine only – there is no compiler in the browser – so the `code/` route is the local installation's privilege; the expression route and the per-unit override work in both.

DRAFT

State directories are restartable snapshots. The steady solution is written to `converged/` in the same layout as `0/`, and only when the run converged: the directory's name is a contract. Each unit's internal state (a column's stage profile, a reactor's axial profile, a crystal size distribution) is written beside the streams under `internalStates/`, one file per unit, so a converged directory renamed `0/` restarts the case from its answer. The time-integrated binaries write a time directory at every write interval in the same layout. `iterations/` keeps the numerical history when asked for, and is never physical time. A case can also be *sealed*: `bin/choupo-import` copies the full closure of property records the case consumes into its own `constant/` under a manifest with a checksum per record, and a sealed case runs with the installation catalogue forbidden to it, so a tutorial answers the same on any machine; the seal is computational – it compares parsed content, not bytes, so a comment moved is cosmetic and a value moved is a refusal – and every tutorial in the corpus is sealed.

A plant and its sectors. A large flowsheet may be organised as a plant and its sectors, each sector a folder with its own `system/` and `constant/`; the engine flattens the tree to one flat problem with qualified names, and a sector may declare its own thermodynamic model. The reference case runs five sectors across four thermodynamic models (an electrolyte Pitzer brine, an NRTL liquid–liquid split, a Gibbs reactor and a molecular section) and closes its atom balance on every element; a stream crossing from one model to the next is re-priced there, and the enthalpy step is *accounted* in a model-boundary ledger of its own rather than charged to the unit that received it.

3.3 The browser front-end is a visualiser, on the ParaView pattern

In the usual process-simulator workflow the user *builds* the case in the graphical interface. Choupo does the opposite, on the pattern of OpenFOAM and ParaView: OpenFOAM computes, and ParaView shows what was computed. Choupo computes – natively, or in WebAssembly from the same C++ – and the browser front-end shows the case so that the user understands it. The case is always text dictionaries, authored in an editor or with an agent; the front-end loads them, draws them, runs the engine on them and shows what the engine published, and nothing it shows is computed by the front-end itself. That stance was settled before the first screen was drawn and is the front-end's design rule: it is a viewer, not a block editor and not an IDE. Pressing F1 on any view opens the guide at the section that derives it.

What it shows, in the version cited here: the flowsheet, with each stream coloured by the phase the engine resolved for it; the run log, streamed live while the engine runs; the three global balances – mass, atoms, energy – drawn from the engine's own reports and never recomputed (the rule since the day the front-end's own first-law sum was found to disagree with the engine's by an order of magnitude); a plant and its sectors, each sector openable as a case of its own; the stream table and each unit's results and profiles; the specification sheets under `design/` – one dictionary per sized item, regenerated every run, each value with its unit and its design basis and an `assumed` line naming what the case did not declare – drawn as a datasheet by equipment kind; the Explorer – the compound catalogue, an inspector with provenance marks, an estimate form, and property surfaces (T-x-y, γ, P^{sat}, LLE, ternary, Gibbs maps, Bjerrum, steam tables, transport, phase envelopes, Hildebrand parameters), with an Evidence view that draws measured points against candidate models being merged; the EduTools; and the plant layout in 3D – the plot plan with the case's declared separations and zones, pipes routed orthogonally on a rack, the plot area, roads and site buildings, a fence with gates, a truck route walked against a declared vehicle, the GMP building interior – all declared by the case, checked by the engine and drawn by the front-end, each item by its equipment kind on the envelope the engine published and, today, sitting at grade (foundations and the final piping detail are future work). The next step on the development line, in the sense of Section 7's evolution paragraph and not of the cited version, is the instrumented flowsheet: a P&ID layer declared over the PFD – loops, transmitters, control valves, ISA tags – validated by the engine, drawn by the front-end as a second view, and used as the input from which `choupoCtrl` runs the same case with its control loops.

DRAFT

3.4 What Choupo takes from OpenFOAM, and where it differs

Choupo's case organisation is inspired by OpenFOAM®[1] [1], the open-source C++ toolbox for continuum mechanics whose plain-text dictionaries and case directories we regard as one of the most elegant designs in scientific software. Table 2 maps the concepts one to one and says, for each, whether Choupo does the same thing or something different and why. **Choupo is an independent project: it is not part of, affiliated with, endorsed by or sponsored by OpenCFD Ltd or the OpenFOAM Foundation, and no OpenFOAM source code was adapted or reused in it.** *This offering is not approved or endorsed by OpenCFD Limited, producer and distributor of the OpenFOAM software via www.openfoam.com, and owner of the OPENFOAM® and OpenCFD® trade marks.*

Table 2: OpenFOAM concepts and their Choupo equivalents.

OpenFOAM concept	Choupo equivalent	Same, or differs because
A case is a directory: `system/`, `constant/`, `0/`	the same three directories	Same. `system/` holds control, topology and solver dictionaries; `constant/` the thermophysical system; `0/` the complete initial state.
Hierarchical plain-text dictionaries, `keyword value;`; `controlDict` with `writeInterval`	the same grammar and the same control file	Same. Choupo requires a unit or a dimension set on every dimensional scalar and converts to SI on reading; `writeInterval` drives the time-integrated binaries, and the steady one writes `converged/` once, only on convergence.
A field file carries `internalField` and `boundaryField`	a state view carries the streams (the boundary) and `internalStates/<unit>` (the interior)	Same idea, one level up: a stream is an edge between units, a unit's profile is its interior, and both are needed to restart.
Time directories `0.1/`, `0.2/`	`<t>/` in the dynamic binaries; `iterations/` for numerical history	Same for physical time. Differs: a steady run keeps its iteration history under a separate name, so a directory named by a number always means time.
One application per solver algorithm (`icoFoam`, `simpleFoam`, ...)	one application per *problem class* (five binaries)	Differs: a flowsheet's numerical strategy (Newton, Wegstein, Nelder–Mead, ...) is chosen in the case, so a student changes the method without changing the program.
Run-time selection through registration macros and static tables	explicit `registerBuiltins()` functions called from `main()`	Differs by design: no macro and no static initialiser, so the origin of every model is one readable function; the cost is one line and a rebuild per built-in.
Residual normalisation (`normFactor`)	`solver/Convergence.H`: the same structure with equations in place of cells	Same structure and intent; the header states the three places where the analogy stops (a single-equation system degenerates and falls back to the raw residual, announced).
OpenFOAM computes; ParaView shows what was computed	the engine computes; the browser front-end shows the case and the result	Same pattern (Section 3.3): the case is text, the viewer computes nothing.
Compiled utilities (`blockMesh`, `foamDictionary`, ...)	small scripts and tools in `bin/` (`runCase`, `choupo-init0`, `choupo-lint`, ...)	Differs: Choupo's utilities are mostly scripts around the five binaries.
Default branch is the development line; releases are versioned tags (`OpenFOAM-dev`, `OpenFOAM-13`)	`main` is `Choupo-dev`; a release is an immutable tag `vYYMM`	Same, adopted after checking that project's repositories rather than from memory.

4 Thermophysical data

4.1 A source and a licence for every value

The curated catalogue (`data/standards/`) cites the primary source per value, never an aggregator's arrangement; each record says where its numbers came from and how they were obtained (measured, regressed, estimated), and a record that has not been reviewed, or that comes from the user's private local tier, is announced as such every time a run reads it. Licence classes are decided before a value enters the tree: a dataset with no licence statement is treated as no-grant and kept out; a

[1] OPENFOAM® is a registered trade mark of OpenCFD Limited, producer and distributor of the OpenFOAM software via www.openfoam.com.

DRAFT

NonCommercial grant is kept out, because Choupo is free for commercial use; a pair table does not acquire its courier's licence by being redistributed inside a permissively licensed package. The engine refuses to write into the curated catalogue: new data enters it by a human act of curation, and a gate (defined in Section 5) holds every record to these rules.

4.2 The curation tools, and what a promoted record carries

Curation is a chain of commands, each leaving a file a reader can open. `bin/choupo-thermoml` locates measured data in the open NIST/TRC ThermoML archive – joined to the catalogue by InChIKey, never by name – and extracts a dataset that names its article, its DOI, the archive file and block, the per-point uncertainty and a review status. `choupoProps`' `fitParameters` operation regresses a model's parameters on datasets declared as *fit* and scores them on datasets declared as *held-out*, against an acceptance band declared *before* the fit; `evidencePartition` announces how independent the two roles are; `vleConsistency` tests a dataset's thermodynamic consistency; the `curationDossier` records what was used, excluded and assumed. The fit writes a *proposal* record and promotes nothing: promotion into the catalogue is a human act (`bin/curate/promote-from-dossier`), and a promoted record carries its evidence, its validity span and its verdict, so a gate can re-run its case and refuse a record that drifts from what produced it. Listing 2 is the validation block of one promoted pair, with the sentence that says where its acceptance band came from.

Listing 2: The `validation` block of the promoted NRTL record for 1-propanol + water (`data/standards/parameters/NRTL/1Propanol-water.dat`; the `origin` sentence shortened at `[...]`).

```
validation
{
    verdict     validated;
    heldOut
    {
        points      19;
        aad_pct     0.056538;
        aad_K       0.205310;
        domain_x1   ( 0.058000 0.976000 );
    }
    acceptance
    {
        maxAAD      0.260000;   // per cent
        origin      "declared before the fit, by the ONE rule of C16 [...]: the held-out set's largest stated
     expanded uncertainty (U95 = 0.22 K) plus the catalogue vapour-pressure model's own error at that set's
     pure endpoints (water -0.703 K, 1Propanol +0.051 K; measured with no pair involved), as a percentage of
     the held-out mean T (362.46 K) = 0.2546 %, rounded UP to two significant figures";
    }
}
```

4.3 A worked example: who does what, and a candidate that fails

The example is the acetone + methanol pair, chosen because it is the azeotrope a student meets in two plants of the tutorial corpus and because it is the one of nine regressed pairs that *failed* its test. The division of responsibilities is the point.

The user states the problem: the mixture (acetone, methanol), the property (bubble temperature of the binary at atmospheric pressure, the quantity a distillation column needs), and the conditions (near 101 kPa, the whole composition range).

The agent finds the evidence and organises it. `choupo-thermoml` locates two independent isobaric studies in the archive, one at 101.3 kPa [2] and one at 100 kPa [3], extracts each with its per-point uncertainty (0.1–0.35 K), sets the pure endpoints aside (they test the vapour-pressure record, not the pair) and writes a `propsDict` that declares the larger study as the fit set and the other as held out, two NRTL parameters (b_{ij}, b_{ji}; α fixed at 0.30), uncertainty weighting, and the acceptance band. The band is computed by one rule applied to every pair before any fit runs: the held-out set's largest stated uncertainty plus the catalogue vapour-pressure model's own error at the pure

DRAFT

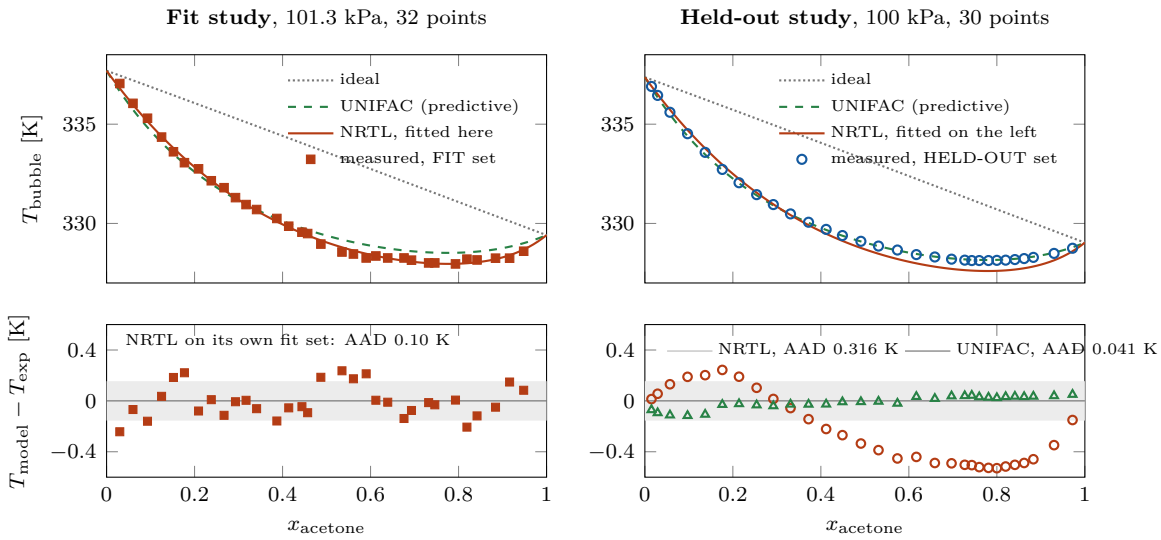


Figure 2: The acetone + methanol regression, drawn from the run's own files. Top: bubble-point curves of three models at each study's pressure, with the study the NRTL pair was fitted to (left, filled squares) and the study it was held out on (right, open circles) – two laboratories, two articles, no author in common. Bottom: residuals per point; the grey band is the acceptance band declared before the fit (±0.155 K). The fitted pair reproduces its own study to 0.10 K and misses the other by 0.316 K; UNIFAC, which saw neither, does the mirror image (0.34 K on the fit study, 0.041 K on the held-out one). The verdict is `notValidated`, and the pair was not promoted.

ends, as a percentage of the held-out mean temperature – here 0.047 %, 0.155 K. The agent then runs the fit.

The engine records what was used and excluded (the two datasets with DOI, archive file, checksum and review status, the endpoints set aside), freezes a fingerprint of the partition before the regression starts, announces that neither dataset has been read back against its article, fits by Levenberg–Marquardt (and again from a second start, landing on the same pair), reports the in-sample residual, and computes the held-out verdict against the declared band. It also runs three bubble-point scans – ideal, UNIFAC, the fitted NRTL – at each study's pressure and overlays both studies on them.

The interface shows the measurements, the model curves, the residuals per point and the limits of use: Figure 2, drawn from the run's own files.

The student reads the dictionaries and the dossier, compares the alternatives on the figure and justifies a choice.

What the student sees in Figure 2 is a candidate that fits its training data well and fails its test. The fitted NRTL pair reproduces the study it was fitted to with an rms of 0.13 K and an average deviation of 0.10 K; on the other study its average deviation is 0.316 K (0.096 %), twice the band declared before the fit, so the verdict is `notValidated` and the record stays in the case rather than entering the catalogue. UNIFAC, a predictive model that saw neither study, reproduces the held-out study to 0.041 K and misses the fit study by 0.34 K. The two studies disagree with each other by about 0.3 K where each states 0.1–0.35 K; which is right is a question for a curator who reads both articles, not for the fitting tool. Swapping the roles of the two studies after seeing the residuals would be the tuning the declared partition exists to prevent; the engine's fingerprint of the partition, frozen before the fit, is what makes that visible.

The student who must choose a model for the column now has grounds: at these pressures UNIFAC describes the independent measurement better than the regressed pair; both place the azeotrope at $x_{\text{acetone}} \approx 0.78$ at 100 kPa, so a column design is not sensitive to the choice there; and neither has been tested at the 6 atm of a pressure-swing scheme, which the engine will announce as an

DRAFT

extrapolation of the pair's evidence if it is run there. A second candidate in the same corpus fails the other way: the ethanol + cyclohexane pair *passed* its held-out band (0.094 % against 0.41 %) and was still not promoted, because the same parameters predict a liquid–liquid split across the whole of their validity span where both studies report one liquid. A bubble-temperature scan solves for one liquid by construction, so its verdict cannot see a second: *validated* is necessary, not sufficient, and the catalogue gate holds every promoted pair to one liquid across its span.

5 Verification and validation

A *gate*, in this project, is an automated check that runs with the test suite and fails if a specific class of error reappears; each one prints the single line that states what it checked and what it did not. The test suite has several kinds of check, and the useful thing to know is what each kind lets one conclude.

A golden-master pass means "not moved", never "right". Of the 480 runnable cases, 463 carry a reference answer that an earlier Choupo run recorded, and every suite run compares each case with its reference (and screens every case for NaN and infinity, because an exit-code check passes on a silent NaN). A pass proves the answer has not moved; it says nothing about whether it was ever right. A re-recorded reference is therefore treated as a claim: the rows that moved are listed, each with the physical or numerical reason, before anything is re-recorded, and a row that moves for a reason nobody can state is a regression until proven otherwise.

Validation is a small, named subset. Following the distinction in NASA-STD-7009 [4], the project keeps a named list of cases anchored to independent published results, recounted by a gate on every run so a case cannot join or leave it unnoticed. The system-level anchors are the Williams–Otto reactor and its published steady state [5], the IAPWS-IF97 verification tables [6], and the Cavett recycle train on its primary specification (the specification, not a published product table – the case says so). The property-level anchors are mostly aqueous electrolytes: the Hamer–Wu activity coefficients of uni-univalent salts [7], a measured calcite pH [8], Carey–Lewis ethanol–water VLE, and others; where an anchor is the model's own training data, the row says "fit-consistency" and is not counted as validation. The subset holds no rigorous column, no recycle plant and no liquid–liquid flowsheet anchored to a measurement, and today only five cases carry *anchor* rows the suite falsifies by machine; the rest of the table is a claim in prose. Section 6.3 adds the membrane anchors, which are held by their own gates.

Conservation is read from the engine's own reports. Every converged steady run writes three balances – total mass, atoms per element, energy at the plant boundary – whether or not the case asked for them; a case may opt out explicitly, never by forgetting. Two gates read those reports for every steady case in the corpus and never recompute them: the mass gate refuses a plant whose boundary closure misses by more than 0.5 % (252 cases today), and the energy gate requires the first law to close within 1 % of the energy the plant exchanges (105 cases) or within 1 kW where the plant exchanges none (70 cases). Plants that do not close are not hidden: 17 are *pinned* by name with their measured residual, and the pin moves both ways – a residual that grows fails, and one that shrinks also fails, asking for the list to be updated – so the ledger always describes the engine as it is. The denominator is the energy exchanged, not the enthalpy throughput: most of the throughput is the formation datum passing through, and dividing by it made a real gap look small. These gates cover steady runs; the time-integrated reactors' first law has not been measured the same way and is claimed neither way.

When an error escapes, it becomes a gate. Choupo has had its share of errors, and some reached users before the check that now catches them existed. The rule since has been that each escaped error gets a gate that rules out the class it belongs to, verified by sabotage (the defect is

DRAFT

reintroduced on purpose and the gate is observed to fail), with a design record that tells how the class was found. Two stories say what such a gate can and cannot reach.

The dryer that made water. A spray dryer's residual-moisture model is a ratio carried by the dry solid and knows nothing of how much water the feed brought; on the flagship plant it produced more water than it received, at exit zero, while the case's reference answer passed – nothing had been looking at closure. The mass gate above exists because of it. Its reach is exactly the 252 steady cases that publish a mass balance and the 0.5 % band; a plant that loses 0.4 % passes it, and a case the suite does not walk is outside it.

The crystalliser volume in kilomoles. A sizer multiplied the unit's *molar* flow by the residence time and called the product cubic metres, so a crystalliser was costed on a holdup an order of magnitude too large – at exit zero, pinned to four decimals by its own reference answer, because a reference pins what a run *prints*. The design-sheet gate now recomputes sizes from what the case *declared*: a crystalliser sheet's magma volume must equal the declared operating volume, a column's five items (shell, trays, condenser, reboiler, reflux drum) are recomputed from the declaration and the run's indicators, a catalyst charge from the declared density and price. Its reach is those recomputations on those witnesses; it does not say whether a sizing rule is the right rule.

Two more are worth a sentence each. A declared model word the engine did not know used to fall through an if-chain to its default: a one-character slip selected the wrong crystalliser model, and on a spray dryer the wrong chamber model with every published indicator byte-identical, so no reference answer could ever have seen it; every such site now refuses through one function that names the accepted words, and the gate counts the sites. And a gate whose input has disappeared, or whose tool is not installed, must refuse rather than pass: a permanently green check looks exactly like a healthy one, and the project retired one that had been reporting success for weeks after both of its inputs were deleted. The gate manifest lists 239 gates with the claim each one's output makes; Appendix A gives the classes behind the examples in this paper. None of this makes the engine error-free; it makes each class of error, once seen, unable to return without a check failing, and the open debts a list with names and numbers.

Who checks the checkers: every gate is written against a record of the defect, verified by sabotage, and reviewed by a human maintainer before it is integrated; conservation is checked by the engine's own balance reports, which are computed in the reporting layer from the stream states and not by the unit that produced them; and the sabotages themselves are listed in each gate's source. A self-test gate reruns twelve of those sabotages on every suite run and says in its own claim that the other gates are unaudited by it rather than implied sound.

6 Representative examples

6.1 A simple case, to understand the calculation

Listing 3 is an excerpt of the run log of the flash in Listing 1, at the default verbosity. It prints the model it uses, the vapour pressures and the K-values they give, each Newton iteration of the Rachford–Rice equation, and – unprompted, on the very first tutorial – that benzene's vapour pressure was evaluated outside the temperature range of its Antoine correlation. It ends with two blocks: whether the problem solved is the problem posed, and every assumption raised during the run, repeated because a warning a thousand lines above the answer has been delivered but not received. The three balances are written by default; the energy line reads a residual of zero and says that no percentage exists, because this drum declares no duty and exchanges no heat, so there is no scale to divide by.

DRAFT

Listing 3: Excerpt of the run log of the flash (`verbosity 3`); elided lines marked `[...]`.

```
>>>  Unit [0]:  flash01   (type = isothermalFlash)
>>>  thermo: inherited (global package)
  [flash] model rachfordRice  (declared)
[psat] component 'benzene': Antoine evaluated at T = 370 K, OUTSIDE its declared Trange (287.7 354.07).
K-values  (γ = ideal,  φ = idealGas,  evaluated at x = z initially):
  benzene       Psat = 1.6514 bar    K = 1.6514
  toluene       Psat = 0.6735 bar    K = 0.6735
  [outer 0] Rachford-Rice Newton (inner):
    it          V           g(V)         dg/dV          ΔV
    0     0.50000000   -3.75806e-02   -1.87920e-01   -1.99981e-01
    1     0.30001863   7.81978e-04    -1.97366e-01   3.96206e-03
    2     0.30398070   5.66359e-07    -1.97081e-01   2.87374e-06
    3     0.30398357   2.94959e-13    -1.97081e-01   0.00000e+00
[...]
  V/F:           0.303984
  Component         z          x          y          K
  benzene         0.400000  0.333888  0.551373    1.6514
  toluene         0.600000  0.666112  0.448627    0.6735
[...]
  [report] massBalance       (global closure 100.000 %)
  [report] globalEnergyBoundary  (residual -0.0000 kW; NO EXCHANGED-ENERGY SCALE -- percentage unavailable)
  [report] elementBalance    (worst element closure off by 0.0000 %)
[...]
PROBLEM SOLVED == PROBLEM POSED.  No divergence declared or imposed.
ASSUMPTIONS AND CAVEATS -- what this run announced along the way
  EXTRAPOLATIONS AND VALIDITY  (3)
    - component 'benzene': vapour pressure evaluated at T = 370 K, OUTSIDE its declared Trange (287.7 354.07)
-- extrapolated, still returned
```

A student can check the two K-values by hand from the printed vapour pressures, count the Newton steps, and ask whether the answer depends on the extrapolation it was told about. The same case also shows what a refusal looks like. A stream of known component flows is fixed by two intensive variables; a feed that declares a temperature, a pressure *and* a vapour fraction is over-specified, and Listing 4 is what the engine returns when the feed of Listing 1 is given one. The run stops with a non-zero exit status before any computation, quotes the rule, the three values and two remedies, and the message is written to be acted on – by a student or by an agent that wrote the file.

Listing 4: A refusal by name (the feed of Listing 1 with `vaporFraction 0.3;` added); exit status 2.

```
ERROR: stream state 'feed': OVER-SPECIFIED -- declares T = 370 K, P = 100000 Pa AND vaporFraction = 0.3 for a
MIXTURE of 2 components (benzene, toluene).  A stream of known component flows is fixed by TWO intensive
variables (Duhem's theorem); for a mixture (T, P) already fix the vapour fraction, so a third value beside
them is redundant when it agrees with the (T, P) flash and a contradiction when it does not.  Remedies: (1)
delete the `vaporFraction` line and let (T, P) resolve the split -- an unpinned stream MEANS its own
equilibrium at (T, P, z), and the stream table and result JSON show the vapour fraction that resolution gives;
 (2) if the QUALITY is the specification, the state is (P, q) or (T, q) and the other variable must be SOLVED
from it -- this engine does not do that today [...], so declare the T that gives your q, or use a phaseChanger
 to produce the state.  A PURE component keeps `vaporFraction q;`: on its saturation curve (T, P) do not fix
the split.
```

6.2 An integrated process with recycles

The Cavett problem [9] is the field's oldest recycle benchmark: four flash drums in a near-isothermal cascade, three vapour recycles, a sixteen-component feed. The Choupo case runs it on the primary specification of Rosen and Pauls [10, 11] – feed, four stage conditions, topology – with SRK and every k_{ij} zero, declared. Figure 3 follows the case from the engineering question to the interpretation, with the numbers of the run behind this paper.

The example is chosen for what it shows about the paper's claim. Convergence to 4×10^{-9} and a mass closure to four decimals are necessary, and the same run publishes a first law that misses by 88 % of the energy exchanged, localises the gap on two units, and names a stream whose carried phase label disagrees with its state. A student handed this run has a question to work on, and a tool that will not let the question disappear: the residual is pinned in the energy gate's open ledger with its value, and the gate fails the day it grows or the day it closes without the ledger being updated.

DRAFT

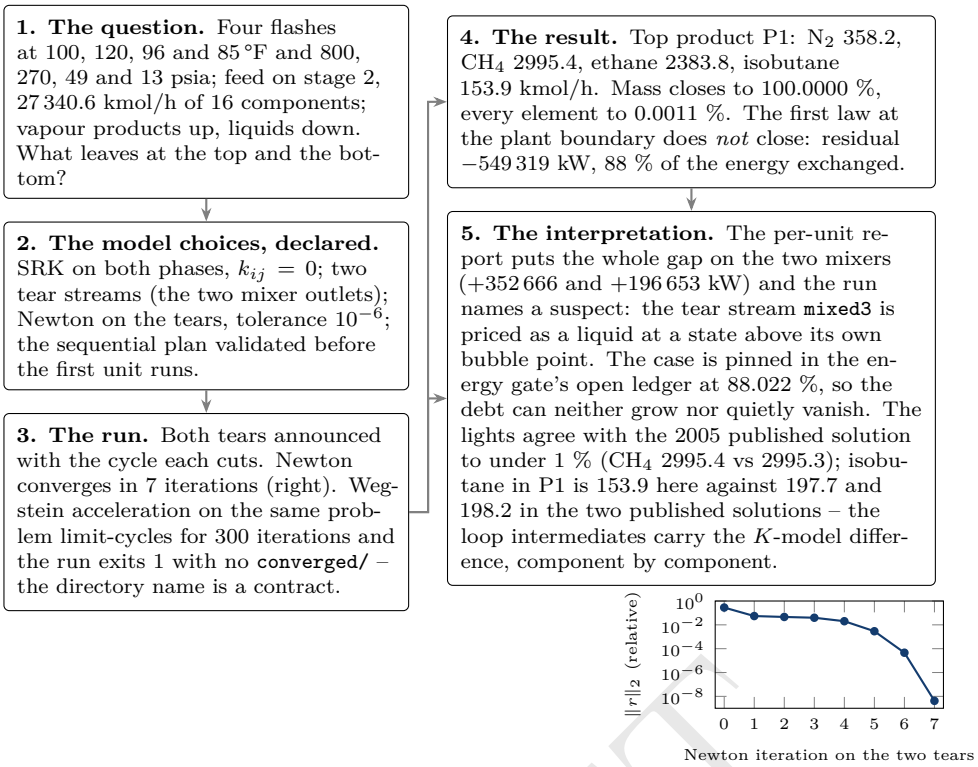


Figure 3: The Cavett recycle train from the engineering question to the interpretation, with the numbers of one run. Mass and atoms close; the first law does not, the run says where, and the gate keeps the number.

6.3 A research application: ions through a nanofiltration membrane

Why does nitrate slip through a nanofiltration membrane that holds chloride, when the two ions have almost the same hydrated size? Choupo's spiral-wound module answers it with the solution-diffusion-electromigration law (`transport SDEM;`): every ion keeps its own permeance, the ions share one electric field fixed by zero current, and the permeate is electroneutral with no parameter the uncoupled law does not already have. The law reduces exactly, for a single salt, to $R = J_v/(J_v + P_s)$ with the ambipolar salt permeance, which is what makes per-ion permeances calibratable from the single-salt tests manufacturers publish. The two witness cases take the NF270 permeances of Table 1 of Fernández de Labastida and Yaroshchuk [12] (CC BY 4.0): with NaCl dominant, chloride is rejected 77 % and nitrate 31 %; with $MgSO_4$ dominant, nitrate's rejection is −15 % – more nitrate in the permeate than in the feed – and the run publishes the membrane potential (34 and 24 mV) so the mechanism is a number rather than a sentence.

What is validated here is stated exactly, because a research reader will ask. The paper's eight figure panels were digitised (106 points, reading error declared per point); the gate drives the engine to each figure's flux and holds it, in the trace limit the authors' analytical fit assumes, to their drawn lines within 2–8 % on every series whose permeances the paper determines. At the paper's own trace concentration of 2×10^{-4} M the engine does *not* reproduce the lines on the sulfate series, and the reason is the paper's own caveat: a trace nitrate at 42 μm/s carries more current than 0.01 M of sulfate at 0.13 μm/s, so the traces relax the field the trace-limit fit assumes they do not touch – pinned as a finding, never tuned. Two of the paper's permeances are bounds, and one drawn line is not the paper's own equation with the paper's own parameter; both are recorded. The measured symbols themselves are compared by nobody: the engine is held to the authors' fit, not to their residual. The same discipline covers the electrodialysis stack, whose predicted limiting current is held to six rows of Table 4 of Geraldes and Afonso [13] within +1 to +12 % (6.9 % average, against the paper's own reported 9 %).

DRAFT

6.4 A comparison with an open-source reference simulator

We compared Choupo with DWSIM [14], an open-source process simulator licensed under the GPL-3.0, on a sample from the library it ships. It is named because it is open: its licence and source were read before anything was relied on, and its maintainers can reproduce every number below. Simulators whose source is closed are not compared, because their models cannot be inspected. The benchmark cases are being integrated into the development line; this section describes them as they stand on their branch, and the findings about the reference below are being offered to its maintainers before this section is made public.

Method. The sample was rewritten as a Choupo case from its published problem – feeds, units, specifications – and no file or parameter of the sample was copied: the samples embed pair parameters from databanks this project excludes, so every property number in the benchmark is Choupo's own or comes from a primary source. The reference numbers are the stored solved state inside the sample file, cited by repository commit and by the build version the file records (8.8.0.0, saved 2024-07-09). Every workaround is stated in the case: the sample specifies both columns by bottoms flow and the two bottoms specifications admit no steady state, so Choupo specifies distillate rates that take what the feed brings; the sample returns the recycle to the condenser stage and Choupo returns it to stage 2 (finding C1 below); a letdown valve is written out between the 6 atm distillate and the 1 atm column.

The case: methanol–acetone pressure-swing distillation. A 40-stage column at 1 atm and a 20-stage column at 6 atm, the high-pressure distillate recycled to the first. Choupo ran it twice: with the acetone–methanol NRTL pair of Section 4.3 – not validated, used case-locally and announced as such, and 50 K beyond its evidence in the 6 atm column, which the run announces as well – and with predictive UNIFAC. Table 3 gives the results beside the reference. The two Choupo variants agree with each other to 0.006 in every distillate composition and 1.5 kW in every duty; they part only on the 6 atm azeotrope (x_{MeOH} 0.465 against 0.524), which no stream of this plant reaches. The duties agree with the reference to between −0.9 and +1.5 %; the distillate compositions differ by 0.017 and 0.022 (7 % relative), and both follow from the flows rather than from an azeotrope: the sample stores no azeotrope, neither column top is pinched at these specifications, and the reference's own flows do not close by more than these differences (findings R1 and R2). The acetone product leaves the 6 atm column pure, so its temperature involves no activity model: the −2.05 K is the difference between the two programs' vapour-pressure correlations at that pressure. Agreement in this one case is evidence for this exercise only; two independent implementations with different vapour-pressure correlations and different pair evidence that agreed to the last digit would suggest one had been tuned to the other.

Table 3: Methanol–acetone pressure swing: the reference's stored state against two Choupo runs (its own regressed NRTL pair; predictive UNIFAC). The reference's stored state closes its material balance to +0.14 % and its first law to +1.47 kW of about 4640 kW exchanged; both Choupo runs close every unit's mass balance and the plant first law to +0.079 kW.

	reference	Choupo NRTL	Choupo UNIFAC	NRTL − ref.
LP condenser duty [kW]	2575.5	2609.9	2609.6	+1.3 %
LP reboiler duty [kW]	2563.3	2600.8	2600.6	+1.5 %
HP condenser duty [kW]	2054.7	2041.6	2040.2	−0.6 %
HP reboiler duty [kW]	2076.5	2058.0	2056.5	−0.9 %
LP distillate x_{MeOH}	0.2412	0.2243	0.2290	−0.017 (−7 %)
HP distillate x_{MeOH}	0.3066	0.2844	0.2904	−0.022 (−7 %)
azeotrope at 1 atm, x_{MeOH} (model)	–	0.2222	0.2247	–
azeotrope at 6 atm, x_{MeOH} (model)	–	0.4651	0.5236	–
acetone product T [K] (pure, 6 atm)	394.95	392.89	392.89	−2.05 K
methanol product T [K] (pure, 1 atm)	337.90	337.70	337.70	−0.20 K

What was found on each side, with the version it refers to. On the reference side, read from one stored file (build 8.8.0.0, repository commit `1abf72d1b`): the two bottoms specifications admit no

DRAFT

steady state (two products of 0.5 mol/s from a feed of 0.9986 mol/s, +0.14 %); the recycle stream as attached carries 1.47 % more than the stream it copies while the recycle block reports convergence after one iteration; the distillate stream is stored 1.98 K below its own condenser stage. These are facts about one stored file, measured on its own numbers; they bound how finely the comparison can resolve, and they are being offered to the maintainers as an issue. On Choupo's side, in the engine version cited here (development line, `feb112a00`): (C1) the multi-feed column accepts a feed on its condenser stage and then omits it from that stage's balance – the default mass-balance report flagged the plant at 109 % closure, but the run exited normally when it should have refused; (C2) the column's "total condenser" publishes the vapour in equilibrium with the reflux, which is one more equilibrium contact than a total condenser, and Table 3's HP distillate composition depends on it (0.085 in x at the top of the HP profile). The correction of C1 is in integration on a development branch and is not in the cited version; C2 is being analysed against the column's formulation, and any change to it will be shown with every result it moves before anything is re-recorded. A second sample, a four-stage water/acetone/MIBK extraction, is not yet reproduced: Choupo's predictive UNIFAC did not describe the water–MIBK miscibility gap, a liquid–liquid NRTL set was curated from the measured tie lines of Choi and Rhim [15], and with it Choupo's liquid–liquid flash returned one liquid on feeds its own stability test had found unstable, while its extractor published a non-converged cascade with a warning instead of refusing. Both are engine defects; correcting them is scheduled after the column work, and until then that comparison is future work. Even then it will have a limit: the reference's column is adiabatic and cools by 13 K, while Choupo's extractor is isothermal by construction.

7 Discussion, limitations and evolution

Nothing in this paper should be read as a claim of breadth or of proof. Choupo does not aim at parity with industrial simulators: its component catalogue is curated rather than encyclopaedic, it has no equation-oriented solver, and its models cover what its tutorials exercise. What it offers is a calculation that can be checked – by the student, by the agent that prepared it, by a reviewer – and a development line in which an escaped error becomes a permanent check. The limitations we know of are these.

- *A regression pass is not validation*, and the validation subset is small: no rigorous column, no recycle plant and no liquid–liquid flowsheet is anchored to a measurement, and five cases carry machine-checked anchor rows.
- *Open first-law debts are pinned, not closed.* Seventeen plants do not close their first law within the gate's band; the Cavett train of Section 6.2 is one. They are listed with their residuals, and they are open.
- *Some balances have not been measured.* The first law of the time-integrated batch reactor and dynamic CSTR is claimed neither way.
- *A gate's reach is its scope.* The benchmark cases are outside the regression suite, and a gate sees only the cases it walks: the stage-1 feed of Section 6.4 was flagged by the default report but not refused, because no tutorial exercised it.
- *Curated data are mostly unreviewed.* The regressed pairs carry `reviewStatus interim`: their measurements were transcribed from the archive and have not been read back against the articles, and every run that uses one says so. Four records with an encumbered source are pinned awaiting curation rather than silently kept.
- *Known numerical limits* are documented beside the feature they bound: bubble-point distillation through an azeotrope is unstable and the simultaneous method is the documented escape; the liquid–liquid flash of Section 6.4 is being corrected.
- *The pedagogical claim is untested*, as Section 1 says.

DRAFT

Evolution. The near-term work is visible in the repository's own records: integrating the benchmark cases and the column corrections; the evidence view in the browser, which draws every measured point against every candidate model for a component or a pair, with the fit and held-out roles distinct and the verdict in context; a worked lesson that walks the curation example of Section 4.3 page by page; reading the transcribed datasets back against their articles; and widening the pair catalogue by the same declared-partition rule, one pair at a time. A Python interface is not planned: Choupo's outputs are plain-text dictionaries, CSV and JSON, and any language reads them.

8 Availability and reproduction

Choupo is free software under the GNU General Public License, version 3 or later. The source, the tutorials, the guides and the browser application are at `https://github.com/choupo-admin/choupo` and `https://www.choupo.org`; the latest release is `Choupo-2608`, and the default branch carries the development line that this paper describes. `CITATION.cff` in the repository gives the preferred citation. The guides are licensed under CC BY-SA 4.0. Every listing and figure in this paper was produced from a named case in the repository with the commands recorded in the source of this document, so a reader with a build of the cited version can regenerate each one. A short companion article and this white paper are both tied to the version they describe. Zenodo and arXiv deposits of this paper and of the release it cites: to be deposited.

Acknowledgements

Choupo is a registered trade mark of TalentGround Lda. OPENFOAM® is a registered trade mark of OpenCFD Limited, producer and distributor of the OpenFOAM software via www.openfoam.com; Choupo is an independent project, not affiliated with OpenCFD Ltd or the OpenFOAM Foundation. DWSIM is named only as the open-source software compared in Section 6.4; no endorsement by its authors is implied. Funding and institutional support: none to declare at the time of this draft.

Statement on the use of AI assistance. Substantial parts of Choupo's implementation, documentation and tutorial corpus, and the drafts of this paper, were produced with the assistance of AI coding agents working under written briefs and human curation. Every part of the project has been reviewed, corrected and accepted by its maintainers; the authors have checked this paper's claims against the software and answer for its content. Copyright in Choupo belongs to its named human contributors.

DRAFT

References

[1] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, 12(6):620–631, 1998. doi: 10.1063/1.168744.

[2] X. Chen, B. Yang, A. A. Abdeltawab, S. Al-Deyab, G. Yu, and X. Yong. Isobaric vapor–liquid equilibrium for acetone + methanol + phosphate ionic liquids. *Journal of Chemical & Engineering Data*, 2015. doi: 10.1021/je5007373.

[3] A. V. Orchillés, P. J. Miguel, E. Vercher, and A. Martínez-Andreu. Ionic liquids as entrainers in extractive distillation: Isobaric vapor–liquid equilibria for acetone + methanol + 1-ethyl-3-methylimidazolium trifluoromethanesulfonate. *Journal of Chemical & Engineering Data*, 2007. doi: 10.1021/je0603170.

[4] National Aeronautics and Space Administration. Standard for models and simulations. Technical Report NASA-STD-7009, NASA, 2008.

[5] T. J. Williams and R. E. Otto. A generalized chemical processing model for the investigation of computer control. *Transactions of the American Institute of Electrical Engineers, Part I: Communication and Electronics*, 79(5): 458–473, 1960.

[6] Wolfgang Wagner, J. R. Cooper, A. Dittmann, J. Kijima, H.-J. Kretzschmar, A. Kruse, R. Mareš, K. Oguchi, H. Sato, I. Stöcker, O. Šifner, Y. Takaishi, I. Tanishita, J. Trübenbach, and Th. Willkommen. The IAPWS industrial formulation 1997 for the thermodynamic properties of water and steam. *Journal of Engineering for Gas Turbines and Power*, 122(1):150–184, 2000. doi: 10.1115/1.483186.

[7] W. J. Hamer and Y.-C. Wu. Osmotic coefficients and mean activity coefficients of uni-univalent electrolytes in water at 25 °C. *Journal of Physical and Chemical Reference Data*, 1(4):1047–1100, 1972.

[8] L. N. Plummer and E. Busenberg. The solubilities of calcite, aragonite and vaterite in CO_2–H_2O solutions between 0 and 90 °C, and an evaluation of the aqueous model for the system $CaCO_3$–CO_2–H_2O. *Geochimica et Cosmochimica Acta*, 46(6):1011–1040, 1982.

[9] R. H. Cavett. Application of numerical methods to the convergence of simulated processes involving recycle loops. In *Proceedings of the American Petroleum Institute*, volume 43, page 57, 1963.

[10] E. M. Rosen and A. C. Pauls. Computers & Chemical Engineering, 1 (1977) 11 – the FLOWTRAN solutions of the Cavett problem, as cited in the Choupo case header, 1977.

[11] E. M. Rosen. Revisiting the Cavett problem. CACHE News, Fall 2005 (the article's full title also names a commercial simulator; shortened here), 2005.

[12] Marc Fernández de Labastida and Andriy Yaroshchuk. Nanofiltration of multi-ion solutions: Quantitative control of concentration polarization and interpretation by solution-diffusion-electro-migration model. *Membranes*, 11(4): 272, 2021. doi: 10.3390/membranes11040272. CC BY 4.0.

[13] Vítor Geraldes and Maria Diná Afonso. Limiting current density in the electrodialysis of multi-ionic solutions. *Journal of Membrane Science*, 360(1–2):499–508, 2010. doi: 10.1016/j.memsci.2010.05.054.

[14] Daniel Wagner Oliveira de Medeiros. DWSIM: an open-source chemical process simulator. https://github.com/DanWBR/dwsim, 2024. GPL-3.0 (README and COPYING read at commit 1abf72d1b6b41d3e9a8cc770d3cc4e8fc76e5766, 2026-07-17); sample files carrying BuildVersion 8.8.0.0.

[15] J. S. Choi and J. N. Rhim. Liquid–liquid equilibria for ternary systems with solvent – water – acetone. *Hwahak Konghak (Journal of the Korean Institute of Chemical Engineers)*, 24(3):185–194, 1986.

A Gate inventory: the classes behind the examples

The generated gate manifest lists 239 gates, each with the claim its own output makes; Table 4 gives the classes of error named in this paper, the gate that keeps each from leaving a run in silence, and the design record under `docs/design/` that tells how the class was found.

Table 4: Classes of error named in this paper and the gate that holds each. Each gate's claim is the line its own output marks as one (`generated/gateManifest.json`).

The class this gate removes	Gate	Record
A plant that does not conserve mass, read from the engine's own report on every steady case	`check_mass_closure`	`a-dryer-that-ended-wetter-than-it-started.md`
A plant whose atoms do not close, element by element	`check_element_balance`, `check_default_reports`	`what-a-passing-suite-does-not-say.md`
A first law that does not close, or an open one that grows or quietly closes	`check_energy_closure`	`what-a-passing-suite-does-not-say.md`
A unit computing with a state its streams do not carry	`check_feed_thermal_state`	`the-state-a-unit-computes-with.md`
A stream declared by three variables where two fix it	`check_overspecified_stream`	`a-stream-is-fixed-by-two-variables.md`
A declared word the engine does not know, silently taken as the default	`check_model_default_registered`	`three-silences-at-exit-zero.md`
An approximation nobody authorised	`check_problem_divergence`	`problem-divergence-contract.md`
An equipment size that cannot be recomputed from the declaration	`check_design_sheet`	`a-basis-nobody-stated-and-a-volume-in-kilomoles.md`
A promoted pair that drifts from the evidence and verdict that produced it, or predicts a second liquid in its validity span	`check_regressed_pairs`	`binary-pairs-from-open-measurements.md`
A membrane law that violates electroneutrality or the single-salt identity, or drifts from the digitised reference figures	`check_sdem`	`electroneutrality-without-a-new-parameter.md`
A value whose licence does not allow redistribution	`check_source_licence`	`the-licence-of-a-sigma-profile.md`
A write into the curated catalogue by a computation	`check_standards_write_guard`	`a-rule-enforced-in-one-place.md`
A validation case that joins or leaves the named subset unnoticed	`check_validation_subset`	`../architecture/verification-and-validation.md`
A check that passes because it could not run	`runtests_verdict.py`, `check_gate_selftest`	`destructive-gate-contamination.md`
Code that builds natively and not for the browser	`check_wasm_dialect`	(CLAUDE.md, 2026-08-27)

DRAFT